

基于PagedAttention的大型语言模型服务的高效内存管理

摘要： 高吞吐量地服务大型语言模型（Large Language Models, LLMs）需要一次批处理足够多的请求。然而，由于每个请求的键-值缓存（key-value cache, 键值缓存）内存非常庞大且动态涨落，现有系统难以应对。如果管理不当，这部分内存会因为碎片化和冗余复制而严重浪费，限制可批处理的请求数量。为了解决这一问题，我们提出了一种注意力算法 **PagedAttention（分页注意力算法）**，其灵感来自操作系统中经典的虚拟内存分页技术。在此基础上，我们构建了LLM服务系统 **vLLM**，实现了：(1) 键值缓存内存近乎零浪费；(2) 在请求内部及跨请求灵活共享键值缓存，从而进一步降低内存占用。我们的评估显示，在保持同等延迟水平的情况下，**vLLM** 将各种流行LLM的吞吐量提升了 2~4 倍，相较当前最新系统（如 FasterTransformer 和 Orca）。在更长序列、更大模型和更复杂解码算法的场景下，这一提升更加显著。**vLLM** 的源代码已在 GitHub 项目公开：<https://github.com/vllm-project/vllm>。

1 引言

大型语言模型（Large Language Models, 大型语言模型, LLM）的出现（如 GPT 系列【5, 37】和 PaLM【9】）催生了诸多新应用，例如编程助手【6, 18】和通用聊天机器人【19, 35】等，这些应用正开始深刻影响我们的工作与日常生活。许多云服务公司【34, 44】正在竞相将这些应用作为托管服务提供。然而，运行这些应用成本高昂，需要大量硬件加速器（如 GPU）。据近期估计，处理一个 LLM 请求的成本可能是传统关键词查询的 10 倍【43】。鉴于成本如此高，提高 LLM 服务系统的吞吐量——从而降低每次请求的成本——变得愈发重要。

*本文多位作者贡献相等。

LLM 的核心是自回归 Transformer 模型【53】。该模型基于输入（提示词）和此前已生成的输出序列（由单词组成，称为**token**）逐次生成新的单词（token）。对于每个请求，这一高成本的生成过程会重复进行，直到模型输出终止符。这个序列化的生成过程使得整个工作负载受制于内存带宽，GPU 的计算能力未被充分利用，从而限制了服务吞吐量。

图1 左：在 NVIDIA A100 GPU（40GB 显存）上服务一个参数量为130亿的 LLM 时的内存布局。模型参数（灰色）在服务全程常驻于GPU显存中。键值缓存（红色）内存会在每次服务请求时动态分配和释放。一小部分内存（黄色）用于瞬态的激活值。右：**vLLM** 平滑了现有系统【31, 60】中键值缓存内存激增的曲线，从而显著提升了服务吞吐量。

通过将多个请求一起批处理可以提高吞吐量。然而，要在一个批次中处理大量请求，必须高效管理每个请求的内存空间。例如，图1（左）展示了在一块40GB NVIDIA A100 GPU上，为一个130亿参数的 LLM 提供服务时的内存分布情况。大约65%的内存分配给了模型权重，这些权重在服务过程中是静态的。接近30%的内存用于存储请求的动态状态。对于 Transformer 模型而言，这些状态包括注意力机制关联的键和值张量，通常称为 **键值缓存（KV cache, 键值缓存）**【41】。键值缓存表示生成新输出 token 时从先前 token 获取的上下文。剩下一小部分内存用于存放其他数据，包括激活值（即执行 LLM 时临时创建的张量）。由于模型权重是恒定的，且激活值仅占用极小的

GPU 内存，键值缓存的管理方式对确定最大批处理规模至关重要。如果管理不善，如图1（右）所示，键值缓存内存可能严重限制批处理大小，从而限制 LLM 的吞吐量。

在本研究中，我们观察到现有的 LLM 服务系统【13, 12†2023a】在键值缓存内存管理方面表现不佳。主要原因是它们将每个请求的键值缓存存储在**连续的内存空间**中，因为大多数深度学习框架【16, 17】要求张量以连续方式存储。然而，与传统深度学习任务中的张量不同，键值缓存具有独特的特性：它会随着模型生成新 token 动态增长和收缩，其生命周期和长度在生成开始前无法预知。这些特性导致现有系统采用的连续存储策略在以下两个方面存在显著低效：

图2：在§6.2的实验中，不同 LLM 服务系统中各种内存浪费的平均占比。*Token states* 表示实际用于存储 token 状态的部分，*Reservation* 表示预留未用部分，*Internal frag.* 表示内部碎片，*External frag.* 表示外部碎片。

首先，现有系统【13, 12†2023a】会产生**内部碎片**和**外部碎片**。为了在连续空间中存储一个请求的键值缓存，这些系统预先分配一块与请求最大长度（例如2048个 token）相对应的连续内存。这会导致严重的内部碎片，因为请求的实际长度可能远小于其最大长度（参见图11）。即使事先知道实际长度，预分配仍然低效：由于在请求生命周期内整块内存都被保留，其他较短的请求无法利用该块中当前未使用的部分。此外，不同请求所需的预分配大小可能不同，也会导致显著的外部碎片。我们的分析（图2）表明，在现有系统中，只有20.4%~38.2%的键值缓存内存用于存储实际的 token 状态。

其次，现有系统无法**利用内存共享的机会**。LLM 服务常常使用高级解码算法，例如并行采样和束搜索（beam search），为每个请求生成多个输出。在这些情况下，一个请求包含多条序列，这些序列的键值缓存可以部分共享。然而，由于每条序列的键值缓存都存储在独立的连续空间中，现有系统无法实现内存共享。

为了解决上述局限，我们提出了 **PagedAttention（分页注意力算法）**。这一注意力算法的灵感来自操作系统解决内存碎片和共享问题的方法：带分页的虚拟内存。PagedAttention 将每个请求的键值缓存划分为多个**块**，每个块包含固定数量 token 的键和值向量。在 PagedAttention 中，各键值块并不一定存储在连续的空间中。因此，我们可以像操作系统的虚拟内存一样，以更灵活的方式管理键值缓存：可将块视为“页”、将 token 视为“字节”、将请求视为“进程”。这种设计通过采用相对较小的块并按需分配，减轻了**内部碎片**；由于所有块大小相同，也消除了**外部碎片**。最后，它允许在块粒度上共享内存，无论是在同一请求的不同序列之间，还是在不同请求之间共享。

在本工作中，我们基于 PagedAttention 构建了高吞吐、分布式的 LLM 服务引擎 **vLLM**，实现了键值缓存内存近乎零的浪费。vLLM 采用块级内存管理和与 PagedAttention 协同设计的抢占式请求调度。vLLM 支持多种主流 LLM（如 GPT【5】、OPT【62】、LLaMA【52】等）及不同规模的模型，包括那些单块 GPU 显存无法容纳的模型。我们在各种模型和工作负载上的评测表明，**vLLM** 在保持模型精度完全不受影响的前提下，将 LLM 服务吞吐量提升了2~4倍，相较之前最先进的系统【31, 60】。序列越长、模型越大、解码算法越复杂（参见第4.3节），vLLM 的改进越显著。

总而言之，我们的主要贡献如下：

- 我们识别了 LLM 服务中内存分配所面临的挑战，并量化了这些问题对服务性能的影响。
- 我们提出了 PagedAttention，这是一种可以在非连续分页内存中操作键值缓存的注意力算法，其灵感来自操作系统中的虚拟内存分页技术。
- 我们设计并实现了构建于 PagedAttention 之上的分布式 LLM 服务引擎 vLLM。

- 我们在多种场景下评估了 vLLM，结果显示其性能远超先前的最佳方案（如 FasterTransformer 【31】 和 Orca 【60】 ）。

2 背景

在本节中，我们将描述典型 LLM 的生成与服务过程，以及 LLM 服务中采用的迭代级调度策略。

2.1 基于 Transformer 的大型语言模型

语言模型任务旨在对一个 token 列表 $(x_1, \dots, x_n)$ 建模其概率。由于语言天然具有序列顺序，一般会将整个序列的联合概率分解为条件概率的乘积（即**自回归分解**【3】）：

$$P(x) = P(x_1) \cdot P(x_2 \mid x_1) \cdot \dots \cdot P(x_n \mid x_1, \dots, x_{n-1}). \quad (1)$$

Transformer 【53】 已成为大规模建模上述概率的事实标准架构。Transformer 架构中最重要的组成部分是 **自注意力层**。对于输入隐藏状态序列 $(x_1, \dots, x_n) \in \mathbb{R}^{n \times d}$ ，自注意力层首先对序列中每个位置 i 应用线性变换，得到查询向量、键向量和值向量：

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i. \quad (2)$$

接下来，自注意力层通过将位置 i 的查询向量与它之前所有位置的键向量相乘来计算注意力得分 a_{ij} ，并将这些得分作为权重，对对应的值向量取加权平均以计算输出 o_i ：

$$a_{ij} = \frac{\exp(q_i^{\top} k_j / \sqrt{d})}{\sum_{t=1}^i \exp(q_i^{\top} k_t / \sqrt{d})}, \quad o_i = \sum_{j=1}^i a_{ij} v_j. \quad (3)$$

除了公式 (3) 中的计算，Transformer 模型中的其他所有组件——包括嵌入层、前馈层、层归一化 【2】、残差连接 【22】、输出 logits 计算，以及公式 (2) 中的查询、键和值向量变换——全部都是对各位位置独立执行的，即以 $y_i = f(x_i)$ 的形式逐位置应用。

2.2 LLM 服务与自回归生成

在训练完成后，LLM 通常被部署为条件生成服务（例如完成文本的 API 【34】 或聊天机器人 【19, 35】）。对 LLM 服务的一个请求会提供一系列输入提示 token，LLM 服务则根据公式 (1) 生成一系列输出 token。我们将提示 token 列表与输出 token 列表的串联称为**序列**。

由于有公式 (1) 的分解，LLM 只能一次生成一个新 token，并且每个新 token 的生成过程依赖该序列中所有先前已生成的 token，特别是依赖它们的键和值向量。在这一序列生成过程中，先前 token 的键和值向量通常会被缓存以供未来 token 生成之用，这就是所谓的**键值缓存**。需要注意的是，一个 token 的键值缓存取决于它之前出现的所有 token。这意味着，即便同一个 token 出现在序列的不同位置上，其键值缓存也会不同。

给定一个请求的提示词，LLM 服务的生成计算可以分为两个阶段：

- **提示阶段 (prompt phase)**：该阶段将整个用户提示 $(x_1, \dots, x_n)$ 作为输入，计算第一个新 token 出现的概率 $P(x_{n+1} \mid x_1, \dots, x_n)$ 。在此过程中，模型也会生成提示部分每个 token 的

键向量 $k_1, \dots, k_n$ 和值向量 $v_1, \dots, v_n$ 。由于提示 token $x_1, \dots, x_n$ 都是已知的，因此提示阶段的计算可通过矩阵-矩阵乘法并行完成。因此，该阶段能够高效利用 GPU 固有的并行能力。

- **自回归生成阶段 (autoregressive generation phase)**：该阶段顺序生成其余的新 token。在迭代步骤 t ，模型将上一次生成的单个 token x_{n+t} 作为输入，并利用键向量 $k_1, \dots, k_{n+t}$ 和值向量 $v_1, \dots, v_{n+t}$ 计算下一个 token 出现的概率 $P(x_{n+t+1} \mid x_1, \dots, x_{n+t})$ 。注意，第1到第 $n+t-1$ 个位置的键和值向量已在之前的迭代中缓存，此次迭代只需计算新的键和值向量 k_{n+t} 和 v_{n+t} 。当序列长度达到设定的最大值（由用户指定或由模型限制）或遇到终止符（如“<eos>”）时，该阶段停止。由于存在数据依赖，不同迭代间的计算无法并行，而且这一阶段主要使用矩阵-向量乘法，效率较低。因此，该阶段严重低效地利用 GPU 计算资源，是单次请求延迟的主要部分。

图3：现有系统中的键值缓存内存管理示意图。存在三种类型的内存浪费——预留未用 (reserved)、内部碎片 (internal fragmentation) 和外部碎片 (external fragmentation) ——导致其他请求无法充分利用内存。每个内存槽中的 token 短语表示其键值缓存。注意，相同 token 在不同位置上的键值缓存内容可能不同。

2.3 面向 LLM 的批处理技术

通过批处理多个请求可以提升 LLM 服务中的计算资源利用率。由于同一批次的请求共享模型权重，在一个批次内移动模型权重的开销可以被多个请求分摊，当批量足够大时，这部分开销相较计算开销可以忽略不计。然而，为 LLM 服务进行批处理并非易事，原因有二。首先，请求到达的时间可能不同。一个简单的批处理策略要么会让先到的请求等待后来的请求，要么将后来的请求延迟到前面的处理完毕后再启动，这都会导致显著的排队延迟。其次，不同请求的输入和输出长度可能有巨大差异（见图11）。一种直接的批处理方式是请求的输入和输出进行填充，使它们长度相等，但这会浪费 GPU 计算和内存资源。

为了解决上述问题，已经有细粒度的批处理机制被提出，如**蜂窝批处理** (cellular batching) 【29】和**迭代级调度** 【13】。与在请求层面进行批处理的传统方法不同，这些技术在迭代层面工作。在每次迭代后，已完成生成的请求会从批次中移除，同时加入新的请求。因此，一个新请求只需等待一个迭代即可开始处理，而不必等整个批次完成。此外，借助专门的 GPU 内核，这些技术避免了对请求输入和输出进行填充。通过减少排队延迟并消除填充所带来的低效，细粒度批处理机制显著提升了 LLM 服务的吞吐量。

3 LLM 服务中的内存挑战

尽管细粒度批处理减少了计算浪费，使请求能够更灵活地批处理在一起，但一次能够批处理的请求数量仍受限于 GPU 内存容量，特别是用于存储键值缓存的空间。换言之，服务系统的吞吐量受内存限制。要突破这一内存瓶颈，需要解决内存管理中以下挑战：

- **巨大的键值缓存**：键值缓存大小会随着请求数的增加而快速增长。例如，对于有130亿参数的 OPT 模型【20】来说，单个 token 的键值缓存需要约 800 KB 的空间，其计算方法为：2（键向量 + 值向量） $\times$ 5120（隐藏状态维度） $\times$ 40（层数） $\times$ 2（每个FP16的字节数）。由于 OPT 最多可生成 2048 个 token，一次请求的键值缓存内存最高可达 1.6 GB。当今 GPU 的内存容量在几十GB量级。即使把所有可用内存都用于键值缓存，也只能容纳几十个请求。此外，如图2所示，低效的内存管理会进一步减少可批处理的请求数量。并且从当前趋势看，GPU 的计算速度增长快于内存容量【17】。例如，从 NVIDIA A100 到 H100，浮点运算性能提升了超过2倍，但 GPU 显存仍维持在最高80GB。因此，我们认为内存将成为日益重要的瓶颈。

- **复杂的解码算法：** LLM 服务为用户提供多种解码算法可选，每种算法对内存管理的复杂性有不同影响。例如，当用户希望从单一输入提示生成多个随机样本（这是编程建议等应用中的典型用例【18】），提示部分的键值缓存可以在多个生成序列间共享，在我们的实验中提示部分占总键值缓存内存的12%【6.3节】，通过共享可以减少内存占用。另一方面，在自回归生成阶段，不同样本结果的上下文和位置不同，键值缓存**不应共享**。键值缓存能在多大程度上共享，取决于采用的具体解码算法。在更复杂的算法如束搜索（beam search）【49】中，不同 beam（候选序列）之间可以共享更大部分的键值缓存（最多可节省55%的内存，见第6.3节），并且共享模式会随着解码过程推进而动态变化。
- **未知的输入和输出长度调度：** 发送到 LLM 服务的请求在输入和输出长度上差异很大。内存管理系统需要能够适应各种提示长度。此外，随着请求在解码阶段生成的输出长度不断增长，其键值缓存占用的内存也随之扩张，可能会耗尽新请求或现有请求继续生成所需的可用内存。系统需要做出调度决策，例如将某些请求的键值缓存从 GPU 内存中删除或换出（swap out）。

3.1 现有系统的内存管理

由于当前大多数深度学习框架【33, 39】中的算子都要求张量存储在连续内存中，以往的 LLM 服务系统【31, 60】也将单个请求的键值缓存视为跨越不同 token 位置的连续张量。由于 LLM 的输出长度不可预测，这些系统根据请求可能的最大序列长度为每个请求静态分配一块内存区域，而不考虑请求实际的输入长度或最终输出长度。

图3 描绘了两个请求的情况：请求A的最大可能序列长度为2048，请求B的最大为512。现有系统采用的整块预分配方案存在三类主要的内存浪费来源：为未来 token 保留的槽位（reserved slots）、由于为潜在最大序列长度过度预留而产生的内部碎片，以及源自内存分配器（如伙伴分配算法）的外部碎片。外部碎片所对应的内存存在服务请求前就已确定不会用于生成 token；内部碎片在请求结束采样后才意识到未被使用。两者都是纯粹的内存浪费。即使预留的内存最终会被使用，但是在请求整个持续期间一直保留这部分空间——特别是当预留空间很大时——会占据原本可以用于处理其他请求的内存空间。我们在图2中展示了实验中平均的各类内存浪费比例，可以看到在之前的系统中实际有效利用的内存占比最低仅有20.4%。

图4：vLLM 系统概览。

虽然已有研究提出采用**内存紧凑**（compaction）【54】作为减少碎片的潜在方案，但是在对性能高度敏感的 LLM 服务系统中执行内存紧凑并不现实，因为键值缓存规模非常庞大。即使使用内存紧凑，现有内存管理系统中为每个请求预留整块空间的方式，仍然阻碍了解码算法所特有的内存共享机会。

4 方法

在本工作中，我们开发了一种新的注意力算法 **PagedAttention（分页注意力）**，并构建了 LLM 服务引擎 **vLLM**，以应对第3节列出的问题。vLLM 的体系结构如图4所示。vLLM 采用中心调度器（centralized scheduler）来协调多个分布式 GPU 工作线程的执行。**键值缓存管理器**借助 PagedAttention 有效地以分页方式管理键值缓存。具体来说，键值缓存管理器通过中心调度器发出的指令，管理 GPU 工作节点上的物理键值缓存内存。

接下来，我们将在第4.1节描述 PagedAttention 算法。在此基础上，我们阐述键值缓存管理器的设计（第4.2节），以及它如何配合 PagedAttention 工作（第4.3节）。然后，我们展示该设计如何有效支持多种解码方法（第4.4节）并处理可变长度的输入输出序列（第4.5节）。最后，我们说明 vLLM 的系统设计如何在分布式环境下运行（第4.6节）。

4.1 PagedAttention

为了解决第3节中的内存挑战，我们引入 **PagedAttention**——这是一种受操作系统经典分页思想【25】启发的注意力算法。与传统注意力算法不同，PagedAttention 允许将连续的键和值向量存储在 **非连续的内存空间**。具体而言，PagedAttention 将每条序列的键值缓存划分为若干 **键值块**。每个块包含固定数量 token 的键向量和值向量，我们将这个固定数量称为**块大小** B 。设**键块** $K_j = (k_{(j-1)B+1}, \dots, k_{jB})$ ，**值块** $V_j = (v_{(j-1)B+1}, \dots, v_{jB})$ 。则公式 (3) 中的注意力计算可以转化为按块进行的如下计算：

$$A_{ij} = \frac{\exp(q_i^{\top} K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^{\top} K_t / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j \cdot A_{ij}^T. \quad (4)$$

其中， $A_{ij} = (a_{i,(j-1)B+1}, \dots, a_{i,jB})$ 表示在第 j 个键值块上的整行注意力得分向量。

在注意力计算过程中，PagedAttention 的内核会分别识别并提取所需的**不同键值块**。图5 给出了 PagedAttention 的示例：键和值向量分散在三个块中，这三个块在物理内存中并不连续。每次计算时，内核取查询 token “forth” 的查询向量 q_i ，与一个块中的键向量 K_j 相乘（例如对于块0，键向量对应“Four score and seven”），计算得到注意力得分 A_{ij} ；随后，将 A_{ij} 与该块的值向量 V_j 相乘，以获得最终的注意力输出 o_i 。

图5：PagedAttention 算法示意图，其中注意力的键和值向量在内存中以非连续的块存储。

总之，PagedAttention 算法允许将键值块存储在非连续的物理内存中，从而使 vLLM 能够采用更灵活的分页内存管理方式。

4.2 键值缓存管理器

vLLM 内存管理的核心思想类似于操作系统中的 **虚拟内存**【25】。操作系统将内存划分为固定大小的页，并将用户程序的逻辑页映射到物理页。连续的逻辑页可以对应非连续的物理内存页，从而使用户程序感知到的内存是连续的。此外，物理内存空间不需要预先全部保留，OS 可以按需动态分配物理页。vLLM 借鉴虚拟内存的理念来管理 LLM 服务中的键值缓存。在 PagedAttention 的支持下，我们将键值缓存组织为固定大小的键值块，就像虚拟内存中的页一样。每个请求的键值缓存由一系列逻辑键值块构成，随着新 token 及其键值缓存的生成而从左到右填充。最后一个键值块中尚未填充的位置会预留给将来生成。对于 GPU 工作节点，一个**块引擎**会在 GPU 显存中分配一块连续空间，并将其划分为物理键值块（对于换出到 CPU 的情况，在 CPU 内存中也执行类似操作，详见第4.5节）。键值缓存管理器还维护**块表**——即每个请求的逻辑块与物理键值块之间的映射关系。每项块表条目记录逻辑块对应的物理块编号以及该块已填充的位置数。将逻辑块与物理块分离使 vLLM 可以在不预留完整连续空间的情况下动态增长键值缓存，从而消除现有系统中的绝大部分内存浪费（如图2所示）。

图6：vLLM 中块表映射示意。

4.3 基于 PagedAttention 的解码过程与 vLLM 实现

下面我们结合图6，通过一个示例逐步阐释 vLLM 如何执行 PagedAttention 并在解码过程中管理内存：

1. **初始映射**：类似 OS 的虚拟内存，vLLM 不需要一开始就为可能的最大生成序列长度预留所有内存。相反，它仅预留满足提示计算阶段所需的键值块。在图6的例子中，提示包含7个 token，因此 vLLM 将前两个逻辑键值块（块0和块1）映射到两个物理键值块（分别是物理块7和物理块1）。在“预填充（prefill）”步骤，vLLM 使用常规的自注意力算法（如【13】）计算提示 token 和第一个输出 token 的键值缓存。随后，

vLLM 将前4个 token 的键值缓存存储到逻辑块0，将接下来的3个 token 存储到逻辑块1。剩余的槽位则预留给随后的自回归生成阶段。

2. **第一次自回归解码：** 在自回归解码的第一次迭代中，vLLM 在物理块7和1上使用分页注意力算法生成新 token。由于最后一个逻辑块（块1）中还有空余槽位，新生成的键值缓存被存储在该逻辑块剩余空间中，同时块表中该块已填充位置的计数被更新。
3. **第二次自回归解码：** 在第二次迭代时，由于最后一个逻辑块已填满，vLLM 需要为新生成的键值缓存添加一个新的逻辑块；vLLM 分配了一个新的物理块（物理块3）与之对应，并将这一映射记录到块表中。

整体而言，对于每次解码迭代，vLLM 首先选择一组候选序列进行批处理（详见第4.5节），并为新出现的每个逻辑块分配相应的物理块。然后，vLLM 将当前迭代中所有待处理的输入 token（对于提示阶段的序列即全体提示 token，对于生成阶段的序列即最新生成的 token）**拼接**成一个大的序列，送入模型计算。在模型执行过程中，vLLM 使用分页注意力内核访问此前已按照逻辑块形式存储的键值缓存，并将新生成的键值缓存写入物理键值块中。将多个 token 存储于单个键值块（块大小 > 1 ）可以使分页注意力内核在更长的位置范围内并行处理键值缓存，从而提升硬件利用率并降低延迟。然而，较大的块大小也会增加内存碎片。我们将在第7.2节研究块大小对性能的影响。

此外，vLLM 会随着更多 token 及其键值缓存的生成，动态地为新的逻辑块分配物理块。由于所有块都严格从左到右填充，且只有当之前所有块满载后才分配新块，vLLM 将每个请求导致的所有内存浪费限制在一个块的范围内。因此，vLLM 能有效利用几乎全部内存，如图2所示。这使得内存中可以容纳更多请求进行批处理——从而提高吞吐量。当一个请求完成生成后，其占用的键值块即可释放，供其他请求的键值缓存使用。在图7中，我们示例展示了 vLLM 管理同时处理两个序列（请求）的内存情况。两个序列的逻辑块被映射到 GPU 工作节点预留空间内的不同物理块。两个序列相邻的逻辑块在 GPU 物理内存中无需相邻，各物理块的空间都能被两个序列高效利用。

图7：vLLM 同时存储两个请求的键值缓存示例。

4.4 对其他解码场景的适用

第4.3节展示了 PagedAttention 和 vLLM 如何处理基本的解码算法，例如对单一用户提示生成单条输出序列的贪心解码（greedy decoding）和随机采样。这一节中，我们讨论 vLLM 在更复杂解码场景中的通用适用性，这些场景具有更复杂的内存访问模式，并提供更多内存共享的机会。

并行采样： 在基于 LLM 的编程助手等应用中【6, 18】，LLM 可能针对同一个输入提示生成多个采样输出，供用户从不同候选中选择自己喜欢的结果。迄今为止，我们隐含地假设每个请求仅生成单一路径的序列。接下来，本论文其余部分将假定更一般的情况：每个请求可能生成多条序列。

对于并行采样，一个请求包括多个共享相同输入提示的采样输出，因此这些输出序列的提示部分键值缓存可以共享。借助分页注意力和分页内存管理，vLLM 能轻松实现这一共享，从而节省内存。

图8 显示了并行解码生成两个输出的示例。由于两个输出共享相同的提示，我们在提示阶段只为提示的状态预留了一份空间；两个序列提示部分的逻辑块映射到相同的物理块：两个序列的逻辑块0和块1都映射到物理块7和物理块1。由于单个物理块可以映射到多个逻辑块，我们为每个物理块引入了一个**引用计数**。在该例中，物理块7和物理块1的引用计数均为2。在生成阶段，由于两个输出将采样出不同的输出 token，需要分别存储各自的键值缓存。vLLM 对需要被多个序列修改的物理块采用**写时复制**（copy-on-write）的机制，粒度为块级，类似于操作系统虚拟内存中

的写时复制技术（例如进程 fork 时）。具体而言，在图8中，当样本A1需要写入其最后一个逻辑块（逻辑块1）时，vLLM 检查到对应的物理块（物理块1）的引用计数大于1；于是它分配一个新的物理块（物理块3），指示块引擎将物理块1的内容复制到新的物理块3，并将原物理块1的引用计数减至1。随后，当样本A2需要写入物理块1时，由于此时引用计数已降为1，因此A2可以直接将新生成的键值缓存写入物理块1。

总而言之， vLLM 能够在多个输出采样间共享大部分提示键值缓存，仅最后一个逻辑块例外——对于最后一个块，采用写时复制机制管理。通过在多个采样序列间共享物理块，内存使用可显著降低，尤其对于长输入提示而言。

图8：并行采样示例。

束搜索：在机器翻译等 LLM 任务中【59】，用户期望模型输出前 k 个最合适的翻译。**束搜索（Beam search）**【49】被广泛用于从 LLM 解码最可能的输出序列，因为它降低了对全部可能输出遍历的计算复杂度。该算法依赖 beam width 参数 k ，表示每一步保留的最高分候选数。在解码过程中，beam search 对当前每个候选序列尝试扩展所有可能的下一个 token，利用 LLM 计算它们的概率，然后在 $k \cdot |\mathcal{V}|$ 个新候选（ $|\mathcal{V}|$ 为词表大小）中选出概率最高的 k 条序列，作为新的候选。

与并行解码不同，束搜索不仅可以在所有候选序列间共享最初的提示块，还可以在其他块上实现共享，而且共享模式会随着解码过程推进而动态变化，这类似于操作系统中通过多次 fork 创建的进程树。图9 展示了 vLLM 管理 beam search 的一个示例，beam width $k=4$ 。在虚线所示迭代之前，每个候选序列已经使用了4个完整的逻辑块。所有 beam 候选共享第一个块0（即提示）。候选3从第二个块开始就与其他候选不同。候选0~2 共享前三个块，并在第四块开始各自分叉。在后续迭代中，概率最高的4个候选序列全部源自原候选1和2。由于原候选0和3不再处于前4名，其对应的逻辑块被释放，对应物理块的引用计数减小。vLLM 会释放所有引用计数降至0的物理块（如块2、4、5、8），然后 vLLM 分配新的物理块（块9~12）来存储新的候选序列生成的键值缓存。此时，所有候选序列共享块0、1、3；候选0和1 共享块6，而候选2和3 进一步共享块7。

以往的 LLM 服务系统在 beam 间需要频繁复制键值缓存。例如，在图9所示情形中，虚线之后，候选3 通常需要复制候选2的大部分键值缓存才能继续生成。而 vLLM 的物理块共享大幅减少了这种频繁的内存复制开销。在 vLLM 中，不同 beam 候选的大部分块都可以共享；仅当新生成的 token 仍需写入旧的共享块时（类似并行解码的情况），才应用写时复制机制。这仅涉及复制单个块的数据。

共享前缀：LLM 用户常常会提供任务的（较长）描述，包括说明和示例输入/输出，这被称为 **系统提示（system prompt）**【36】。该描述与实际的任务输入拼接在一起，形成请求的提示。LLM 基于完整提示生成输出。图10 给出了一个示例。此外，可以通过提示工程进一步调整共享前缀，以提高下游任务的准确性【26， 27】。对于这类应用，许多用户提示包含相同的前缀，因此 LLM 服务提供方可以预先计算并存储该前缀的键值缓存，以减少在前缀上重复计算的浪费。在 vLLM 中，实现这一点非常方便：服务提供方可以预留一组物理块用于存放预定义的共享前缀（类似 OS 为各进程共享库所做的处理）。对于含有共享前缀的用户输入提示，只需将其逻辑块映射到已缓存的物理块（并将最后一个块标记为写时复制）。这样，提示阶段的计算仅需对用户自己的任务输入执行即可。

图9：束搜索示例。

混合解码方法：前文讨论的各种解码方法在内存共享和访问模式上各不相同。然而，vLLM 可以同时高效地处理具有不同解码偏好的请求，这是现有系统无法做到的。这是因为 vLLM 通过一个通用的映射层（将逻辑块翻译为物理块）屏蔽了不同序列之间复杂的内存共享。LLM 模型及其执行内核对于每个序列只看到物理块 ID 列表，而无需处理序列间的共享模式。相比之下，现有系统无法高效地将具有不同采样要求的请求混合批处理；而 vLLM 的方法拓宽了不同请求共同批处理的机会，从而提高了系统整体吞吐量。

图10：机器翻译任务共享前缀示例。示例改编自文献【5】。

4.5 调度与抢占

当请求流量超过系统容量时，vLLM 必须对请求进行优先级排序。在 vLLM 中，我们对所有请求采用先进先出（FCFS）的调度策略，以保证公平并防止某些请求一直得不到服务。当需要抢占请求时，vLLM 确保最早到达的请求先完成服务，而最新到达的请求最先被抢占。

LLM 服务面临一个独特的挑战：LLM 的输入提示长度变化很大，输出长度在生成前也不可预知，取决于输入提示和模型本身。随着请求数量增加及其输出增长，vLLM 可能耗尽 GPU 上用于存储新生成键值缓存的物理块。此时，vLLM 需要解决两个经典问题：(1) 应该淘汰（evict）哪些块？(2) 如果被淘汰的块需要再次使用，如何恢复它们？通常，页面置换算法使用启发式方法预测未来最晚使用的块并将其淘汰。在我们的场景中，我们知道一个序列的所有块会被一起访问，因此我们实现了“要么全部淘汰，要么都不淘汰”的策略。也就是说，对于某一序列，要么将其全部键值块都淘汰出GPU内存，要么一个也不淘汰。此外，同一请求内的多个序列（例如一个束搜索请求中的多个 beam 候选）被作为一个**序列组**共同调度。由于这些序列之间可能共享内存，在一个序列组内的序列总是同时被预empt或同时重新调度。对于第二个问题（如何恢复淘汰的块），我们考虑以下两种机制：

- **换出（Swapping）**：这是大多数虚拟内存实现采用的经典技术，即将淘汰的页复制到磁盘交换区。在我们的场景中，我们将淘汰的块复制到 CPU 内存。如图4所示，除了 GPU 块分配器，vLLM 还包括一个 CPU 块分配器，用于管理换出到 CPU 内存的物理块。当 vLLM 耗尽了可供新 token 使用的空闲物理块时，它会选择一组序列进行淘汰，并将它们的键值缓存转移到 CPU。请求被预empt并块被淘汰后，vLLM 会暂停接受新请求，直到所有被预empt的序列完成。某个请求一旦完成，其占用的块被释放，而被预empt序列的块则被从 CPU 内存调回 GPU，序列继续处理。需要注意的是，在此设计下，换出到 CPU 的块总数不会超过 GPU 内存中用于键值缓存的块总数，因此 CPU 上的交换空间有一个上限，即不超过 GPU 为键值缓存分配的内存量。
- **重计算（Recomputation）**：在这种机制下，当被预empt的序列被重新调度时，直接重新计算其键值缓存。需要注意，重计算的延迟可能显著低于最初计算时的延迟，因为在解码时生成的 token 可以与原始用户提示串联成新的提示——这些 token 在所有位置的键值缓存都可以通过一次提示阶段计算生成。

交换和重计算两种方案的性能取决于 CPU 内存与 GPU 显存之间的带宽，以及 GPU 的计算能力。我们将在第7.3节比较交换和重计算的速度。

4.6 分布式执行

许多 LLM 的参数规模超过了单块 GPU 的显存容量【5, 9】。因此需要采用**模型并行**【28, 63】将模型切分到多块 GPU 上，并以并行方式执行。这就要求内存管理器能够处理**分布式内存**。vLLM 能够支持Transformer模型常用的 Megatron-LM 风格张量模型并行策略【47】来适应分布式场景。该策略遵循“单程序多数据”（SPMD）的执行计划，将线性层按块划分，各 GPU 通过 all-reduce 操作持续同步中间结果。具体来说，注意力算子沿着多头维度划分，每个 SPMD 进程负责多头注意力中一部分注意力头的计算。

我们观察到，即使采用模型并行，每个模型分片仍然处理相同的一组输入 token，因此它们需要用于相同位置的键值缓存。因此，vLLM 在中心调度器中实现了一个全局的键值缓存管理器（如图4所示）。不同 GPU 工作节点共享这个管理器，以及逻辑块到物理块的映射。这一统一的映射使各 GPU 节点能够根据调度器提供的物理块执行模型。尽管每个 GPU 节点具有相同的物理块 ID 集合，但每个节点只存储自己负责的注意力头对应的那部分键值缓存。

在每个迭代步骤中，调度器首先为批次中的每个请求准备包含输入 token ID 以及块表的控制消息，并将该消息广播给所有 GPU 工作节点。随后，GPU 节点根据收到的 token ID 开始执行模型计算。在注意力层，各 GPU 节点依据控制消息中的块表读取键值缓存。在执行过程中，各 GPU 节点无需调度器介入，直接通过 all-reduce 等通信原语同步中间结果【47】。最后，GPU 节点将本次迭代中采样得到的新 token 返回给调度器。总而言之，各 GPU 节点在内存管理上无需相互同步：它们只需在每次解码迭代开始时接收调度器发送的全部内存管理信息及当步的输入数据即可。

5 实现

vLLM 是一个端到端的服务系统，包含基于 FastAPI【15】的前端以及 GPU 上的推理引擎。前端拓展了 OpenAI API【34】接口，允许用户针对每个请求自定义采样参数，如最大序列长度、束宽 k 等。vLLM 引擎由约8.5K行 Python 代码和2K行 C++/CUDA 代码实现。我们使用 Python 实现了包括调度器和块管理器在内的控制组件，同时针对关键操作（如 PagedAttention）开发了自定义的 CUDA 内核。在模型执行方面，我们使用 PyTorch【39】和 Transformers【58】框架实现了 GPT【5】、OPT【62】、LLaMA【52】等主流 LLM。我们使用 NCCL【32】库在分布式 GPU 节点间进行张量通信。

5.1 内核级优化：由于 PagedAttention 引入了现有系统中不易高效支持的内存访问模式，我们开发了若干 GPU 内核来优化这些操作。(1) **融合reshape和块写入：**在每一层 Transformer 中，新生成的键值缓存需要拆分成块，并重排为针对块读取优化的内存布局，然后根据块表指定的位置写入显存。为尽量减少内核启动的开销，我们将这些步骤融合为单个内核实现。(2) **融合块读取和注意力计算：**我们改造了 FasterTransformer【31】的注意力内核，使其能够根据块表读取键值缓存，并即时完成注意力计算。为确保内存访问对齐，我们为每个块分配一个 GPU warp 来读取。(3) **融合块复制：**写时复制机制触发的块复制操作，可能涉及不连续的多个块。如果直接使用 cudaMemcpyAsync API，会产生大量小数据搬运调用。为降低开销，我们实现了一个 GPU 内核，可将不同块的复制操作批量执行，在单次内核调用中完成。

5.2 支持多种解码算法：vLLM 使用三个关键方法——fork、append 和 free——来实现多种解码算法。**fork** 方法从已有序列创建新序列；**append** 方法向序列末尾添加新 token；**free** 方法删除已完成的序列。例如，在并行采样中，vLLM 使用 fork 方法从单一路径的输入序列派生出多个输出序列。在每轮迭代中，它对这些序列调用 append 添加新 token，对于达到终止条件的序列则调用 free 删除。同样的策略也应用于束搜索和前缀共享场景。我们相信未来的新解码算法也可以通过这几个方法的组合得到支持。

6 评估

在本节中，我们在多种工作负载下评估 vLLM 的性能表现。

图11：ShareGPT (a) 和 Alpaca (b) 数据集中输入和输出长度分布。

6.1 实验设置

模型和服务器配置：我们选用 OPT【62】模型（参数量分别为13B、66B、175B）和 LLaMA【52】模型（参数量13B）进行评测。13B和66B是在LLM应用中很流行的规模【38】；175B相当于著名的 GPT-3【5】模型的规模。所有实验均在 Google Cloud Platform 的 A2 实例上进行，硬件为 NVIDIA A100 GPU（40GB显存）。模型规模和服务器配置的细节见表1。

工作负载：我们基于 ShareGPT [51] 和 Alpaca [50] 数据集构造工作负载。这两个数据集包含真实 LLM 服务中的输入和输出文本。ShareGPT 数据集是用户分享的 ChatGPT [35] 对话集合；Alpaca 数据集是通过 Self-Instruct 方法 [57] 由 GPT-3.5 生成的指令数据集。我们对数据集中的文本进行分词，并使用得到的输入和输出长度来合成客户端请求。如图11所示，ShareGPT 数据集的输入提示平均长度是 Alpaca 数据集的 8.4 倍，输出平均长度是 Alpaca 的 5.8 倍，且方差更大。由于数据集中没有时间戳信息，我们使用不同请求速率的泊松分布生成请求到达时间。

基线1：FasterTransformer。 FasterTransformer [31] 是一个以优化低延迟为目标的分布式推理引擎。由于 FasterTransformer 本身没有内置调度器，我们实现了一个自定义调度器，采用类似 Triton [30] 等已有服务系统的动态批处理机制。具体来说，我们根据 GPU 内存容量为每组实验设置一个尽可能大的最大批处理大小 B 。调度器每次从最早到达的请求中取最多 B 个，将它们组成批次发送给 FasterTransformer 处理。

基线2：Orca。 Orca [60] 是当前为优化吞吐量而设计的 LLM 服务系统。由于 Orca 尚未公开发布，我们自行实现了一个版本。我们假设 Orca 使用 **伙伴分配算法** 来确定存储键值缓存的内存地址。基于对输出空间预留程度的不同假设，我们实现了三个版本的 Orca：

- **Orca (Oracle)**：假设系统事先知道请求实际生成的输出长度。这代表 Orca 能达到的上限性能，但在实际中无法实现。
- **Orca (Pow2)**：假设系统对输出空间的预留至多为实际输出长度的2倍。例如，如果真实输出长度是25，则预留32个位置。这相当于将预留长度向上取到最接近的2的幂次。
- **Orca (Max)**：假设系统始终为每个请求预留模型允许的最大序列长度（即2048个 token）。

关键指标：我们关注服务**吞吐量**指标。具体而言，在不同请求速率的工作负载下，我们测量各系统**归一化延迟**，即每个请求端到端延迟与其输出长度的比值 [60]。一个高吞吐量的服务系统应当在高请求到达率下仍能保持低归一化延迟。在大多数实验中，我们使用1小时长度的请求序列进行评测。对于 OPT-175B 模型，由于成本限制，我们使用了15分钟长度的请求序列。

图12：使用 OPT 模型在 ShareGPT 和 Alpaca 数据集上进行单序列生成的结果。

图13：在 ShareGPT（请求率2 req/s）和 Alpaca（请求率30 req/s）负载下，服务 OPT-13B 模型时平均批处理的请求数量。

6.2 基础采样性能

我们在三种模型和两个数据集上评估 vLLM 在**基础随机采样**（每个请求只生成单一输出）的性能。图12 第一行显示了在 ShareGPT 数据集上的结果。曲线表明，随着请求到达率增加，请求延迟初期缓慢上升，但随后突然激增。这是因为当请求速率超过服务系统容量时，请求队列长度会不断增加，请求延迟也随之无限增长。

在 ShareGPT 数据集上，相较 Orca (Oracle)，vLLM 在保持相近延迟的情况下能支持 1.7~2.7 倍更高的请求到达率；相较 Orca (Max)，这一提升为 2.7~8 倍。这是因为 vLLM 的分页注意力高效管理了内存使用，从而能比 Orca 批量处理更多请求。例如，如图13(a)所示，对于 OPT-13B 模型，在相同时间点 vLLM 比 Orca (Oracle) 同时处理的请求数多出 2.2 倍，比 Orca (Max) 多出 4.3 倍。与 FasterTransformer 相比，vLLM 能承受高达 22 倍的更高请求率，这是因为 FasterTransformer 没有细粒度调度机制，并且内存管理效率和 Orca (Max) 类似低下。

图12 第二行和图13(b)展示了 Alpaca 数据集上的结果，趋势与 ShareGPT 数据集相似。唯一的例外是图12(f)：在这个场景下，vLLM 相较 Orca (Oracle) 和 Orca (Pow2) 的优势不那么明显。这是因为针对 OPT-175B 模型的服务器配置（参见表1）提供了足够大的 GPU 内存用于存储键值缓存，同时 Alpaca 数据集的序列较短。在这种环境下，尽管 Orca (Oracle) 和 Orca (Pow2) 的内存管理存在低效，但也能够批量处理大量请求。因此，各系统的性能瓶颈转为计算（compute-bound）而非内存（memory-bound）。

图14：在 Alpaca 数据集上使用 OPT-13B 模型进行并行生成和束搜索的结果。

6.3 并行采样与束搜索

我们评估了 PagedAttention 在两种常用采样方法下实现内存共享的有效性：并行采样和束搜索。在并行采样中，一个请求的所有并行序列可以共享提示的键值缓存。图14 第一行显示，随着需要采样的序列数增加，vLLM 相比各 Orca 基线的优势也更大。类似地，图14 第二行展示了不同 beam 宽度下束搜索的结果。由于束搜索允许更多共享，vLLM 表现出更大的性能优势。对于 OPT-13B 模型和 Alpaca 数据集，vLLM 相比 Orca (Oracle) 的性能提升从基础采样的 1.3 倍增至 beam width 为6时的 2.3 倍。

图15 绘制了 **内存节省比例**，计算方式为通过共享节省的块数除以未共享情况下的总块数。在并行采样下，我们实现了6.1%~9.8%的内存节省；在束搜索下，该比例为37.6%~55.2%。在使用 ShareGPT 数据集的相同实验中，并行采样实现了16.2%~30.5%的内存节省，束搜索实现了44.3%~66.3%的节省。

图15：在 Alpaca 工作负载下服务 OPT-13B 模型时，通过共享键值块平均节省的内存比例。

6.4 共享前缀

我们研究了 vLLM 在不同输入提示间共享前缀的有效性，如图10 所示。在模型方面，我们使用多语言的 LLaMA-13B 【52】模型。工作负载方面，我们采用 WMT16 【4】英译德数据集，构造了两个前缀：一个包含1个示例（即 one-shot），另一个包含5个示例（few-shot）。如图16(a)所示，当多个请求共享 one-shot 前缀时，vLLM 的吞吐量比 Orca (Oracle) 高出 1.67 倍。而当共享更多示例（图16(b)所示的 five-shot 前缀）时，vLLM 的吞吐量比 Orca (Oracle) 高出 3.58 倍。

图16：翻译任务负载中输入提示共享公共前缀的性能表现。前缀包括 (a) 1 个示例（80 个 token）或 (b) 5 个示例（341 个 token）。

6.5 聊天机器人

聊天机器人【8, 19, 35】是 LLM 最重要的应用之一。为实现聊天机器人，我们将聊天历史与最新的用户提问串联为提示词，让模型生成回复。我们使用 ShareGPT 数据集合成聊天历史和用户提问。由于 OPT-13B 模型的上下文长度有限，我们截断提示至最后1024个 token，并让模型最多生成1024个 token。我们并未在对话轮次之间保留键值缓存，因为若这么做，会在轮次间占用本可用于其他请求的空间。

图17 显示，vLLM 在可支持的请求到达率上比三个 Orca 基线高出 2 倍。由于 ShareGPT 数据集中包含大量长对话，大多数请求的输入提示都达到1024 token。受限于伙伴分配算法，Orca 基线会为每个请求输出预留1024 token 的空间，而不论它们如何预测输出长度。因此，三个 Orca 基线的行为几乎相同。相比之下，vLLM 能有效应对长提示词，因为 PagedAttention 解决了内存碎片和预留的问题。

图17：聊天机器人工作负载的性能表现。

7 消融研究

在本节中，我们通过消融实验从多个方面研究 vLLM，并评估我们所做设计选择的影响。

7.1 内核微基准测试

PagedAttention 中动态的块映射会影响涉及存储键值缓存的 GPU 操作性能，例如块的读/写以及注意力计算。与现有系统相比，我们的 GPU 内核（第5节）需要额外访问块表、执行额外的分支判断，并处理批内不同序列长度。正如图18(a)所示，相比高度优化的 FasterTransformer 实现，我们的注意力内核延迟增加了 20%~26%。我们认为这部分开销很小，因为它仅影响注意力算子，而不涉及模型中的其他算子（如线性变换）。尽管存在这部分开销，由于 PagedAttention 的引入，vLLM 在端到端性能上依然显著超越 FasterTransformer（见第6节）。

图18：消融实验结果。

7.2 块大小的影响

块大小的选择会对 vLLM 的性能产生显著影响。如果块过小，vLLM 可能无法充分利用 GPU 的并行能力来读取和处理键值缓存；如果块过大，会增加内部碎片且降低共享概率。

在图18(b)中，我们在固定请求到达率下，使用 ShareGPT 和 Alpaca 负载对不同块大小的 vLLM 性能进行了评估。在 ShareGPT 负载下，块大小在16到128时性能最佳。在 Alpaca 负载下，块大小为16和32表现良好，但更大的块显著降低性能，因为序列长度比这些块大小还短。在实践中，我们发现块大小取16已经足够充分地利用 GPU，并且对于大多数工作负载而言，这一大小足够小，不会导致显著的内部碎片。因此，vLLM 将默认块大小设定为16。

图19：(a) 不同块大小下重计算和换出的开销；(b) 在相同请求率下使用 ShareGPT 负载服务 OPT-13B 模型时，两种方法的端到端性能比较。

7.3 重计算与换出的比较

vLLM 同时支持重计算和换出作为其恢复机制。为了解两者的权衡，我们对比了各自的端到端性能，并对其开销进行了微基准测试，如图19所示。结果表明，当块较小时，换出会产生极高开销。因为较小的块意味着 CPU 与 GPU 之间需要传输的大量小数据，限制了PCIe带宽的有效利用。相比之下，重计算的开销在不同块大小下基本不变，因为重计算并不使用键值块。因此，当块较小时，重计算更高效；当块较大时，换出更高效。不过即使如此，在我们测试范围内重计算的开销从未超过换出延迟的20%。对于中等块大小（16到64），两种方法的端到端性能相当。

8 讨论

将虚拟内存分页技术应用于其他 GPU 工作负载：虚拟内存分页的理念对于 LLM 服务中的键值缓存管理非常有效，因为这类工作负载需要动态内存分配（输出长度事先未知），且性能受 GPU 内存容量限制。然而，这并非适用于所有 GPU 工作负载。例如，在深度神经网络训练中，张量形状通常是静态的，因此内存分配可提前优化。再比如，对于非 LLM 的推理服务，提高内存效率未必会带来性能提升，因为性能主要受计算限制。在这些场景下，引入 vLLM 的技术可能反而由于额外的内存间接访问和非连续块内存带来性能下降。但我们也乐见 vLLM 的技术应用于具有与 LLM 服务类似特性的其他工作负载。

应用虚拟内存分页时的 LLM 特定优化：vLLM 利用 LLM 应用特有的语义，对经典的虚拟内存分页思想进行了重新解读和增强。其中一个例子是 vLLM 所采用的**整体换出策略**（all-or-nothing swap-out），该策略利用了处理一个

请求需要其对应的所有 token 状态都在 GPU 内存的事实。另一个例子是用于恢复被淘汰块的重计算方法，这是在操作系统中不可行的。此外，vLLM 通过将内存访问与其他操作（如注意力计算）对应的 GPU 内核融合，降低了分页带来的内存间接访问开销。

9 相关工作

通用模型服务系统： 近几年模型服务是一个活跃的研究领域，已经有众多系统被提出以应对深度学习模型部署的不同方面。Clipper【11】、TensorFlow Serving【33】、Nexus【45】、InferLine【10】和 Clockwork【20】是一些早期的通用模型服务系统。它们围绕单模型或多模型的服务在批处理、缓存、放置、调度等方面进行了研究。最近，DVABatch【12】引入了多入口多出口的批处理，REEF【21】和 Shepherd【61】提出了服务中的抢占机制，AlpaServe【28】利用模型并行实现统计复用。然而，这些通用系统没有考虑 LLM 推理的自回归特性和 token 状态，导致错失了针对性的优化机会。

Transformer 专用服务系统： 由于 Transformer 架构的重要性，已有大量专门针对其的服务系统被开发。这些系统利用 GPU 内核优化【1, 29, 31, 56】、高级批处理机制【14, 60】、模型并行【1, 41, 60】和参数共享【64】等技术来实现高效服务。其中，与我们的方法最相关的是 Orca【60】。

与 Orca 的比较： Orca【60】中的迭代级调度和 vLLM 的分页注意力是两种互补的技术：两者都旨在提高 LLM 服务的 GPU 利用率、进而提高吞吐量。Orca 通过对请求进行调度和交错执行，使更多请求可以并行处理；而 vLLM 则通过提高内存利用率让更多请求的工作集能够装入内存。通过减少内存碎片并实现共享，vLLM 可以在同一批次中并行处理更多请求，因而相比 Orca 实现了 $2\sim 4\times$ 的加速。实际上，像 Orca 这样精细的请求调度与交错执行会使内存管理更具挑战性，这使得 vLLM 提出的技术更加关键。

内存优化： 计算能力与加速器内存容量之间日益扩大的差距已使内存成为训练和推理的瓶颈。换出技术【23, 42, 55】、重计算【7, 24】及其组合【40】已用于降低训练峰值内存占用。值得注意的是，FlexGen【46】探索了在有限 GPU 内存下通过换出权重和 token 状态来进行 LLM 推理，但它并不面向在线服务场景。OLLA【48】通过优化张量的生命周期和位置来减少碎片，但它未进行细粒度的块级管理，也未针对在线服务。FlashAttention【13】通过分块计算和内核优化降低了注意力计算的峰值内存和 I/O 开销。本文在在线服务背景下引入了一种块级内存管理的新思路。

10 总结

本文提出了 **PagedAttention** —— 一种允许将注意力的键和值存储在非连续分页内存中的新注意力算法，并给出了高吞吐量、具备高效内存管理的 LLM 服务系统 **vLLM**，该系统由 PagedAttention 提供支持。借鉴操作系统的理念，我们展示了如何将虚拟内存、写时复制等成熟技术改造应用于高效管理 LLM 服务中的键值缓存，并处理各种解码算法。实验表明，vLLM 相比最先进系统吞吐量提升了 $2\sim 4$ 倍。

致谢： 我们感谢 Xiaoxuan Liu、Zhifeng Chen、Yanping Huang、各位匿名的 SOSP 审稿人，以及我们的指导人 (shepherd) Lidong Zhou 所提供的富有洞察的反馈。我们的研究部分得到了以下机构的赠款支持：Andreessen Horowitz、Anyscale、Astronomer、Google、IBM、Intel、Lacework、Microsoft、Mohamed Bin Zayed University of Artificial Intelligence、Samsung SDS、Uber、VMware。

References (参考文献，原文)：

- [1] Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, et al. 2022. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. arXiv preprint arXiv:2207.00032 (2022).
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. Layer normalization. arXiv preprint arXiv:1607.06450 (2016).
- [3] Yoshua Bengio, Réjean Ducharme, and Pascal Vincent. 2000. A neural probabilistic language model. *Advances in neural information processing systems* 13 (2000).
- [4] Ondřej Bojar, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, Varvara Logacheva, Christof Monz, Matteo Negri, Aurelie Neveol, Mariana Neves, Martin Popel, Matt Post, Raphael Rubino, Carolina Scarton, Lucia Specia, Marco Turchi, Karin Verspoor, and Marcos Zampieri. 2016. Findings of the 2016 Conference on Machine Translation. In *Proceedings of the First Conference on Machine Translation*. Association for Computational Linguistics, Berlin, Germany, 131–198. <http://www.aclweb.org/anthology/W/W16/W16-2301>
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021).
- [7] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. arXiv preprint arXiv:1604.06174 (2016).
- [8] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. <https://lmsys.org/blog/2023-03-30-vicuna/>
- [9] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. PaLM: Scaling language modeling with pathways. arXiv preprint arXiv:2204.02311 (2022).
- [10] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. 2020. InferLine: latency-aware provisioning and scaling for prediction serving pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing*. 477–491.
- [11] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. 613–627.

- [12] Weihao Cui, Han Zhao, Quan Chen, Hao Wei, Zirui Li, Deze Zeng, Chao Li, and Minyi Guo. 2022. DVABatch: Diversity-aware Multi-Entry Multi-Exit Batching for Efficient Processing of DNN Services on GPUs. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). 183–198.
- [13] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [14] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. 2021. TurboTransformers: an efficient GPU serving system for transformer models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 389–402.
- [15] FastAPI. 2023. FastAPI. <https://github.com/tiangolo/fastapi>.
- [16] Pin Gao, Lingfan Yu, Yongwei Wu, and Jinyang Li. 2018. Low latency RNN inference with cellular batching. In *Proceedings of the Thirteenth EuroSys Conference*. 1–15.
- [17] Amir Gholami, Zhewei Yao, Sehoon Kim, Michael W Mahoney, and Kurt Keutzer. 2021. AI and memory wall. *RiseLab Medium Post* 1 (2021), 6.
- [18] GitHub. 2022. <https://github.com/features/copilot>
- [19] Google. 2023. <https://bard.google.com/>
- [20] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 443–462.
- [21] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. 2022. Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 539–558.
- [22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [23] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the GPU memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1341–1355.
- [24] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. 2020. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *Proceedings of Machine Learning and Systems* 2 (2020), 497–511.
- [25] Tom Kilburn, David BG Edwards, Michael J Lanigan, and Frank H Sumner. 1962. One-level storage system. *IRE Transactions on Electronic Computers* 2 (1962), 223–235.

- [26] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. arXiv preprint arXiv:2104.08691 (2021).
- [27] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. arXiv preprint arXiv:2101.00190 (2021).
- [28] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. arXiv preprint arXiv:2302.11665 (2023).
- [29] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. 2020. Rammer: Enabling holistic deep learning compiler optimizations with rtasks. In Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation. 881–897.
- [30] NVIDIA. [n.d.]. Triton Inference Server. <https://developer.nvidia.com/nvidia-triton-inference-server>.
- [31] NVIDIA. 2023. FasterTransformer. <https://github.com/NVIDIA/FasterTransformer>.
- [32] NVIDIA. 2023. NCCL: The NVIDIA Collective Communication Library. <https://developer.nvidia.com/nccl>.
- [33] Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fangwei Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. 2017. TensorFlow-Serving: Flexible, high-performance ML serving. arXiv preprint arXiv:1712.06139 (2017).
- [34] OpenAI. 2020. <https://openai.com/blog/openai-api>
- [35] OpenAI. 2022. <https://openai.com/blog/chatgpt>
- [36] OpenAI. 2023. <https://openai.com/blog/custom-instructions-for-chatgpt>
- [37] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL].
- [38] LMSYS ORG. 2023. Chatbot Arena Leaderboard Week 8: Introducing MT-Bench and Vicuna-33B. <https://lmsys.org/blog/2023-06-22-leaderboard/>.
- [39] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems 32 (2019).
- [40] Shishir G Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph Gonzalez. 2022. POET: Training Neural Networks on Tiny Devices with Integrated Rematerialization and Paging. In International Conference on Machine Learning. PMLR, 17573–17583.
- [41] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2022. Efficiently Scaling Transformer Inference. arXiv preprint arXiv:2211.05102 (2022).

- [42] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training.. In USENIX Annual Technical Conference. 551–564.
- [43] Reuters. 2023. <https://www.reuters.com/technology/tech-giants-ai-like-bing-bard-poses-billion-dollar-search-problem-2023-02-22/>
- [44] Amazon Web Services. 2023. <https://aws.amazon.com/bedrock/>
- [45] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. 2019. Nexus: A GPU cluster engine for accelerating DNN-based video analysis. In Proceedings of the 27th ACM Symposium on Operating Systems Principles. 322–337.
- [46] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E Gonzalez, et al. 2023. High-throughput Generative Inference of Large Language Models with a Single GPU. arXiv preprint arXiv:2303.06865 (2023).
- [47] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053 (2019).
- [48] Benoit Steiner, Mostafa Elhoushi, Jacob Kahn, and James Hegarty. 2022. OLLA: Optimizing the Lifetime and Location of Arrays to Reduce the Memory Usage of Neural Networks. (2022). <https://doi.org/10.48550/arXiv.2210.12924>
- [49] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. Advances in neural information processing systems 27 (2014).
- [50] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
- [51] ShareGPT Team. 2023. <https://sharegpt.com/>
- [52] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023).
- [53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. Advances in neural information processing systems 30 (2017).
- [54] Jing Wang, Youyou Lu, Qing Wang, Minhui Xie, Keji Huang, and Jiwu Shu. 2022. Pacman: An Efficient Compaction Approach for Log-Structured Key-Value Store on Persistent Memory. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). 773–788.

- [55] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU memory management for training deep neural networks. In Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming. 41–53.
- [56] Xiaohui Wang, Ying Xiong, Yang Wei, Mingxuan Wang, and Lei Li. 2021. LightSeq: A High Performance Inference Library for Transformers. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers. 113–120.
- [57] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-Instruct: Aligning Language Model with Self Generated Instructions. arXiv preprint arXiv:2212.10560 (2022).
- [58] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations. 38–45.
- [59] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. arXiv preprint arXiv:1609.08144 (2016).
- [60] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 521–538.
- [61] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. SHEPHERD: Serving DNNs in the Wild. In 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23). USENIX Association, Boston, MA, 787–808. <https://www.usenix.org/conference/nsdi23/presentation/zhang-hong>
- [62] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. OPT: Open pre-trained transformer language models. arXiv preprint arXiv:2205.01068 (2022).
- [63] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter-and Intra-Operator Parallelism for Distributed Deep Learning. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 559–578.
- [64] Zhe Zhou, Xuechao Wei, Jiejing Zhang, and Guangyu Sun. 2022. PetS: A Unified Framework for Parameter-Efficient Transformers Serving. In 2022 USENIX Annual Technical Conference (USENIX ATC 22). 489–504.
-